

Vesti Transpiler User Manual

April 4, 2026

Contents

1	Introduction	1
2	Structure of Vesti File	2
3	Keywords	2
3.1	<code>docclass</code> keyword	3
3.2	<code>importpkg</code> keywords	3
3.3	<code>startdoc</code> keyword	3
3.4	<code>useenv</code> keyword	3
3.5	<code>beginv</code> and <code>endenv</code> keywords	4
3.6	<code>defun</code> keyword	4
4	Builtins	5
4.1	<code>#label</code> builtin	5
4.2	<code>#eq</code> builtin	5
4.3	<code>#chardef</code> builtin	6
4.4	<code>#mathchardef</code> builtin	6
4.5	<code>#enum</code> and <code>#enum_counter</code> builtins	6
4.6	<code>#get_filepath</code> builtin	7
4.7	<code>#at_on</code> and <code>#at_off</code> builtins	8
4.8	<code>#ltx3_on</code> and <code>#ltx3_off</code> builtins	8
4.9	<code>#textmode</code> and <code>#mathmode</code> builtins	8
5	Lua API	8
6	Source Code of This Document	9

1 Introduction

I used to create several documents using L^AT_EX (or plain T_EX but T_EX is quite cumbersome to write—especially when working with very complex tables or inserting images). Its markdown-like syntax is also not comfortable to use. For example, here is a simple L^AT_EX document:

```
1 % coprime is my custom class. See https://github.com/e0328eric/coprime.
2 \documentclass[tikz, geometry]{coprime}
3
4 \settitle{My First Document}{Sungbae Jeong}{}
5 \setgeometry{a4paper, margin = 2.5cm}
6
7 \begin{document}
8 \section{Foo}
9 Hello, World!
10 \begin{figure}[ht]
11     \centering
12     \begin{tikzpicture}
13         \draw (0,0) -- (1,1);
14     \end{tikzpicture}
15 \end{figure}
```

```

15 \end{figure}
16
17 The code above is a figure using TikZ.
18
19 \end{document}

```

What annoys me most when using it is the ‘\begin’ and ‘\end’ blocks. Is there a way to write something much simpler? This question led me to start this project. Currently, the following code is generated into the $\text{\LaTeX}$ code above (except comments) using vesti:

```

1 % coprime is my custom class. See https://github.com/e0328eric/coprime.
2 \docclass coprime (tikz, geometry)
3
4 \settitle{My First Document}{Sungbae Jeong}{}
5 \setgeometry{a4paper, margin = 2.5cm}
6
7 \startdoc
8
9 \section{Foo}
10 Hello, World!
11 \useenv figure [ht] {
12     \centering
13     \useenv tikzpicture {
14         \draw (0,0) -- (1,1);
15     }
16 }
17
18 The code above is a figure using TikZ.

```

Everywhere in this paper, the symbol $\sqcup$ denotes a “space” when one use Vesti.

2 Structure of Vesti File

Vesti is similar as $\text{\LaTeX}$. Its structure consists with two parts: **preamble** and **main**. Preamble is the place where $\text{\LaTeX}$ documentclass, packages, and several settings are located. Main body is where actual documentation is located. Below figure is the simple Vesti documentation.

```

1 \docclass article (10pt)
2 \importpkg {
3     geometry (a4paper, margin=2.2cm)
4 }
5 \startdoc
6 Hello, Vesti!

```

We will see later, but the very difference with $\text{\LaTeX}$ is that Vesti has its own keywords (keywords are colored with purple). It makes the code readable and it is easier and faster to write the document. The keyword `startdoc` splits the preamble and the main part of the documentation similar with

`\begin{document}` in $\text{\LaTeX}$. However, Vesti does not have the analogous part of `\end{document}`, because almost every $\text{\LaTeX}$ document (99.999% I’m sure) does not have any code below `\end{document}`. For this reason, Vesti automatically ends document when EOF (End Of File) is found.

3 Keywords

Followings are reserved as keywords. In this document, every Vesti keyword has the form like **this**.

<code>beginv</code>	<code>compty</code>	<code>cpfile</code>	<code>defenv</code>
<code>defun</code>	<code>docclass</code>	<code>endenv</code>	<code>importmod</code>
<code>importpkg</code>	<code>importves</code>	<code>startdoc</code>	<code>useenv</code>

Table 1: Keywords in Vesti

3.1 `docclass` keyword

Keyword `docclass` is an analogous of `\documentclass` in $\text{\LaTeX}$. If `docclass` is in the main paragraph, it acts just a normal word. In other words, `docclass` activates only in the preamble. The syntax of `docclass` is following:

`docclass` $\square$ `<class name>` $\square$ (`<arguments>`)

Here, arguments are separated by commas and embraced by `()`. Here are some examples.

- `docclass` $\square$ `article`
- `docclass` $\square$ `article` $\square$ `(10pt)`
- `docclass` $\square$ `article` $\square$ `(10pt, \square twocols)`
- `docclass` $\square$ `article` $\square$ `(10pt,twocols)`

3.2 `importpkg` keywords

Keyword `importpkg` is an analogous of `\usepackage` in $\text{\LaTeX}$. If `importpkg` is in the main paragraph, it acts just a normal word. In other words, `importpkg` activates only in the preamble.

`importpkg` has two different syntax. First one is same as `docclass`.

`importpkg` $\square$ `<pkg-name>` $\square$ (`arguments`)

Here, arguments are separated by commas and embraced by `()`. In the practical case, one should include several packages with options. `importpkg` also supports such case. We will look at an example instead of giving rigorous grammar.

```

1 importpkg {
2   amsmath, amssymb, amsthm,
3   geometry (a4paper, margin=2.2cm),
4 }

```

As one can see, inside of `{}`, several packages can be used together with their options.

3.3 `startdoc` keyword

Keyword `startdoc` tells to Vesti that the main document starts. In the main document, you can also write `startdoc` in the main document. In that case, `startdoc` does nothing.

3.4 `useenv` keyword

As the name implies, keyword `useenv` is an analogous of `\begin{...}` and `\end{...}` pair in $\text{\LaTeX}$. The simplest `useenv` is like this.

`useenv` `center` {
Hello, World! or `useenv` `center` { Hello, World! }
}

As you can see, `useenv center` is the part of `\begin{center}`, and the single `}` is the part of `\end{center}`. Since Vesti knows their pair, one can write a code with several environment, and each pair is properly matched. For instance, above example is written in Vesti like follows. Here, `\useenv` just prints `useenv` in that style.

```

1 useenv figure [ht] {
2   \centering
3   useenv tikzpicture {
4     useenv scope {
5       \path (0,0) node {\vbox{
6         %#\hbox{\tt\useenv center \{}
7         %#\hbox{\tt\obeyspaces Hello, World!}
8         %#\hbox{\tt\obeyspaces\}}
9       }
10    }
11    \path (2.3,0) node {or};
12    useenv scope [shift={(6,0)}] {
13      \path (0,0) node {\tt\useenv center \{ Hello, World! \}};
14    }
15  }
16 }

```

Full syntax about **useenv** is the following.

$$\text{useenv } \sqcup \langle \text{environment name} \rangle \sqcup \langle \text{argument} \rangle^* \sqcup \{ \langle \text{body} \rangle \}$$

where “*” means that the number of $\langle \text{argument} \rangle$ is zero or at least one, and

$$\langle \text{argument} \rangle = \begin{cases} (\langle \text{argument} \rangle) & \text{mandatory arguments} \\ [\langle \text{argument} \rangle] & \text{optional arguments} \end{cases}$$

For instance, below one is a valid Vesti code (environment **foo** is undefined in general). As one can see, spaces cannot exist in between $\langle \text{argument} \rangle$ s.

```

1 useenv foo (asd)(fff)[\ames and \awdsa](askws)[\rrsaa] {
2   foobar
3 }

```

3.5 **beginv** and **endenv** keywords

As the name implies, both keywords **beginv** and **endenv** are analogous of $\backslash\text{begin}\{\dots\}$ and $\backslash\text{end}\{\dots\}$ pair in $\text{\LaTeX}$, respectively. Thus below code

```

1 beginv center
2   asdsad
3 endenv

```

is exactly same as

```

1 useenv center {
2   asdsad
3 }

```

Then why we need **beginv** and **endenv** if we already have **useenv**? The only reason which both **beginv** and **endenv** exist is defining new environment. For instance, one defines a new environment like following.

```

1 defenv foo {beginv minipage(\textwidth)}{endenv}

```

3.6 **defun** keyword

defun is the keyword which makes a $\text{\LaTeX}$ function. Internally, it uses **xparse** package which is automatically included from modern $\text{\LaTeX}2\epsilon$ kernel. The grammar of **defun** is the following.

$$\text{defun } \sqcup [\langle \text{attr} \rangle] ? \sqcup \langle \text{command-name} \rangle \sqcup (\langle \text{param-spec} \rangle) ? \sqcup \{ \langle \text{implementation} \rangle \}$$

Here, `~?` means that this is an optional.

First, let us see some examples for `defun`, and then explain the meaning of each fields.

```
1 \defun foo {Hello, World!}
```

4 Builtin

Vesti also has its own builtin functions, which are prefixed with `#`. One might wonder what distinguishes builtins from keywords. In fact, from the compiler’s internal perspective, there is no real difference. However, in actual language usage, constantly typing the prefix can be somewhat tedious, especially for functions that are commonly used.

From the perspective of language design sometimes desirable to use names that can not be reserved as keywords. For example, Vesti provides a built-in function `#label`, which will be explained later. Since Vesti is a typewriting-oriented language, the word “label” is often used in its ordinary sense rather than in its special semantic meaning within the language.

Followings are reserved as builtin functions.

<code>#at_off</code>	<code>#at_on</code>	<code>#chardef</code>	<code>#def</code>	<code>#enum</code>
<code>#enum_counter</code>	<code>#eq</code>	<code>#get_filepath</code>	<code>#label</code>	<code>#ltx3_off</code>
<code>#ltx3_on</code>	<code>#mathchardef</code>	<code>#mathmode</code>	<code>#noltx3</code>	<code>#picture</code>
<code>#raw_tex</code>	<code>#showfont</code>	<code>#textmode</code>	<code>#undef</code>	

Table 2: Builtins in Vesti

Since `fancyvrb` does not allow verbatim-like commands inside of `Verbatim` environment, builtins are not colored in example codes.

4.1 `#label` builtin

For the compiling issue, if one label some environment, one should write like follows using $\text{\LaTeX}$ function `\label`.

```
1 useenv theorem { \label{eq:1}
2   1 + 1 = 2.
3 }
```

However, such code is not natural. Thus `#label` comes into the place. One can label environments using `#label` builtin like this.

```
1 #label(eq:1) useenv theorem {
2   1 + 1 = 2.
3 }
```

One can also write like the following, which I personally prefer.

```
1 #label(eq:1)
2 useenv theorem {
3   1 + 1 = 2.
4 }
```

4.2 `#eq` builtin

`#eq` is a abbreviation of ‘`useenv equation`’. Actually, below codes are same.

```
1 #label(eq:1)
2 useenv equation {
3   \sum_{n=1}^{\infty} {1//n^2} = {\pi^2//6}.
4 }
```

```

1 #eq(eq:1) {
2   \sum_{n=1}^{\infty} {1//n^2} = {\pi^2//6}.
3 }

```

Since Vesti is used in the mathematical documents mainly, small syntactic sugar is needed, so Vesti introduces `#eq` builtin.

4.3 `#chardef` builtin

`#chardef` defines a text token using the unicode codepoint. Here is the grammar of `#chardef` builtin.

$$\text{\#chardef } \langle \text{unicode-codepoint} \rangle \langle \text{function} \rangle$$

Here, $\langle \text{unicode-codepoint} \rangle$ must be hexadecimal.

4.4 `#mathchardef` builtin

`#mathchardef` defines a math token using the unicode codepoint. Here is the grammar of `#mathchardef` builtin.

$$\text{\#mathchardef } \langle \text{kind} \rangle \langle \text{font-num} \rangle \langle \text{unicode-codepoint} \rangle \langle \text{function} \rangle$$

Here, $\langle \text{unicode-codepoint} \rangle$ must be hexadecimal. Before explaining each parameter, introduce some examples.

- `#mathchardef .opening 0 29d8 \lfoo`
- `#mathchardef .ordinary 0 2202 \diff`

4.5 `#enum` and `#enum_counter` builtins

`#enum` builtin is just an enumerate environment with minimal support of `enumitem` feature. For example,

```

1 #enum {
2   \item aaa
3   \item bbb
4   \item ccc
5 }

```

prints

1. aaa
2. bbb
3. ccc

`#enum` also can be nested like follows.

```

1 #enum {
2   \item aaa
3   \item #enum {
4     \item bbb
5     \item ccc
6   }
7   \item ddd
8 }

```

It prints

1. aaa
2. (a) bbb

(b) ccc

3. ddd

To customize a format of `\item` like `enumitem`, one can write like the following:

```
1 #enum (\Roman*. ) {  
2   \item aaa  
3   \item #enum ({\roman*}**) {  
4     \item bbb  
5     \item ccc  
6   }  
7   \item ddd  
8 }
```

I. aaa

II. i* bbb

ii* ccc

III. ddd

As one can see, `#enum` changes the format with inside of `()`. In Vesti, single `*` changes into `{<enum count>}` (including braces), and consecutive `**` changes into single `*` character. Thus, `\Roman*` changes into `\Roman{enumi}`. and `{\roman*}**` changes into `{\roman{enumii}}*` (because this part is inside of another `#enum`).

Beware that by the parting mechanism, `\roman***` changes into `\roman*{<enum count>}`, and `\roman{*}**` into `\roman*{<enum count>}`, which are invalid arguments for `\roman` L^AT_EX internal command.

`#enum_counter` builtin is a helper to get the name of the current enum counter. For example, if one wants to start a second enum from 3, one can write

```
1 #enum {  
2   \item aaa  
3   \item #enum {  
4     \setcounter{#enum_counter}{2}  
5     \item bbb  
6     \item ccc  
7   }  
8   \item ddd  
9 }
```

and it prints

1. aaa

2. (c) bbb

(d) ccc

3. ddd

4.6 `#get_filepath` builtin

Because of the Vesti internal, if one includes picture by using `\includegraphics` for instance, providing the raw path cause an error from L^AT_EX with “file not found”. `#get_filepath` builtin adjust raw file path into the new relative one which L^AT_EX knows. Here is the example code.

```
1 \includegraphics{#get_filepath(./foo.png)}
```

Internally, `#get_filepath` changes file path into the relative one with respect to `.vesti-dummy`. For example, if `foo.png`, `foo.ves` and `.vesti-dummy` are located in the root directory, then inside of `foo.ves`, `#get_filepath(./foo.png)` will be changed into `../foo.png` because for `.vesti-dummy`, `foo.png` is located outside of it.

4.7 #at_on and #at_off builtins

Both builtins are same as L^AT_EX functions `\makeatletter` and `\makeatother` plus changes Vesti lexer such that `@` character is also can be a L^AT_EX function name. For instance, without using `#at_on` command, below code does not compiles.

```
1 defun @foo (#1) {Hello, #1!}
```

This is because `defun` expects a valid L^AT_EX function name but the character `@` is not a valid one except after the `\makeatletter` function. Since `defun` is a Vesti grammar, there is no way to tell Vesti that `@` is a right one although one uses `\makeatletter`. By using `#at_on`, below code then compiles.

```
1 #at_on
2 defun @foo (#1) {Hello, #1!}
3 #at_off
```

One can not using `#at_off`, but I strongly recommend to match pairs.

4.8 #ltx3_on and #ltx3_off builtins

Those builtins are similar with `#at_on` and `#at_off` pairs but for L^AT_EX3.

4.9 #textmode and #mathmode builtins

Internally, vesti tracks so called `text mode` and `math mode`. In the math mode, for instance, the token `->` is changed into `→` inside of the math mode. The idea is that Vesti changes `->` into `\to`, and so on. But when defining some function using `defun`, for example, one might want to define

```
1 useenv foo {-><-}
```

so that `$(foo)` makes `→←`, one should change the mode. Here, `#mathmode` comes into the place. One should write instead that

```
1 useenv foo {#mathmode{-><-}}
```

5 Lua API

Vesti uses Lua in both building script and inline inside of Vesti code. Below are functions which are baked in Vesti compiler.

compile	download	getCurrentDir	getEngineType
getModule	joinpath	mkdir	parse
ping	print	setCurrentDir	unzip
vestiDummyDir			

Table 3: Lua builtin functions in Vesti

6 Source Code of This Document

Below code was generated by inline lua.

```
1 docclass article (10pt)
2 importpkg {
3     geometry (a4paper, margin = 2.2cm),
4     xcolor,
5     tikz,
6     fancyvrb,
7     amsmath,
8 }
9
10 \title{Vesti Transpiler User Manual}
11 #::
12 local function read_all(path)
13     local f, err = io.open(path, "rb")
14     assert(f, ("cannot open %s: %s"):format(path, err))
15     local data = f:read("*a")
16     f:close()
17     return data
18 end
19 ::#<readAll>
20
21 -- definition of \keyword command
22 defun [!] keyword (m) {{\tt\color{purple}#1}}
23 defun [!] builtin (v) {{\tt\color{blue!65!yellow}\##1}}
24 #::
25 local commands = { "useenv", "defun", "defenv" }
26 for _, command in ipairs(commands) do
27     local s = string.format(
28         "defun %s (s) {{\IfBooleanTF{#1}{\\keyword{%%!-%s-%%}}{\\keyword{%%!-%s-%%} } }",
29         command, command, command
30     )
31     vesti.print(vesti.parse(s))
32 end
33 ::#
34
35 #::
36 function begenv(add_bang, add_space)
37     local name = "begenv"
38     if add_bang then name = name .. "!" end
39     local begenv = "\\keyword{" .. name .. "}"
40     if add_space then begenv = begenv .. " " end
41     vesti.print(begenv, { nl = 0 })
42 end
43
44 function endenv(add_bang, add_space)
45     local name = "endenv"
46     if add_bang then name = name .. "!" end
47     local endenv = "\\keyword{" .. name .. "}"
48     if add_space then endenv = endenv .. " " end
49     vesti.print(endenv, { nl = 0 })
50 end
51 ::#*
52
53 -- space character used in the texbook.
54 #at_on
55 #chardef 2423 \@b
56 defun ob {\kern2pt{\tt\@b}\kern2pt}
```

```

57 #at_off
58
59 startdoc
60 \maketitle
61 \tableofcontents
62
63 \section{Introduction}
64 I used to create several documents using \LaTeX\ (or plain \TeX\ but \TeX\ is
65 quite cumbersome to write-especially when working with very complex tables or
66 inserting images). Its markdown-like syntax is also not comfortable to use. For
67 example, here is a simple \LaTeX\ document:
68
69 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
70 % coprime is my custom class. See https://github.com/e0328eric/coprime.
71 \documentclass[tikz, geometry]{coprime}
72
73 \settitle{My First Document}{Sungbae Jeong}{}
74 \setgeometry{a4paper, margin = 2.5cm}
75
76 \begin{document}
77 \section{Foo}
78 Hello, World!
79 \begin{figure}[ht]
80     \centering
81     \begin{tikzpicture}
82         \draw (0,0) -- (1,1);
83     \end{tikzpicture}
84 \end{figure}
85
86 The code above is a figure using TikZ.
87
88 \end{document}
89 -%}
90
91 What annoys me most when using it is the \lq\verb@\begin@\rq\ and
92 \lq\verb@\end@\rq\ blocks. Is there a way to write something much simpler? This
93 question led me to start this project. Currently, the following code is
94 generated into the \LaTeX code above (except comments) using vesti:
95
96 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
97 % coprime is my custom class. See https://github.com/e0328eric/coprime.
98 +keyword|docclass@ coprime (tikz, geometry)
99
100 \settitle{My First Document}{Sungbae Jeong}{}
101 \setgeometry{a4paper, margin = 2.5cm}
102
103 +keyword|startdoc@
104
105 \section{Foo}
106 Hello, World!
107 +keyword|useenv@ figure [ht] {
108     \centering
109     +keyword|useenv@ tikzpicture {
110         \draw (0,0) -- (1,1);
111     }
112 }
113
114 The code above is a figure using TikZ.
115 -%}
116

```

```

117 | Everywhere in this paper, the symbol \ob\ denotes a \lq\lq space\rq\rq\ when
118 | one use Vesti.
119 |
120 | \section{Structure of Vesti File}
121 | Vesti is similar as \LaTeX. Its structure consists with two parts: {\tt preamble} and
122 | {\tt main}. Preamble is the place where \LaTeX\ documentclass, packages, and
123 | several settings are located. Main body is where actual documentation is located.
124 | Below figure is the simple Vesti documentation.
125 |
126 | useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
127 | +keyword|docclass@ article (10pt)
128 | +keyword|importpkg@ {
129 |     geometry (a4paper, margin=2.2cm)
130 | }
131 | +keyword|startdoc@
132 | Hello, Vesti!
133 | -%}
134 |
135 | We will see later, but the very difference with \LaTeX\ is that Vesti has its
136 | own keywords (keywords are colored with purple). It makes the code readable and
137 | it is easier and faster to write the document. The keyword startdoc splits
138 | the preamble and the main part of the documentation similar with
139 |
140 | -- Don't ask why I chose Q for catcode 0.
141 | %#{\tt\catcode`Q=0 Qcatcode`\=12 \beginQ{documentQ}} in \LaTeX.
142 | However, Vesti does not have the analogous part of
143 | %#{\tt\catcode`Q=0 Qcatcode`\=12 \endQ{documentQ}},
144 | because almost every \LaTeX\ document (99.999\% I'm sure) does not have any code
145 | below %#{\tt\catcode`Q=0 Qcatcode`\=12 \endQ{documentQ}}.
146 | For this reason, Vesti automatically ends document when EOF (End Of File) is
147 | found.
148 |
149 | \section{Keywords}
150 | Followings are reserved as keywords. In this document, every Vesti keyword has
151 | the form like \keyword{this}.
152 | useenv table [ht] {
153 |     \centering
154 |     #::
155 |     local content = read_all("../src/lexer/Token.zig")
156 |
157 |     -- Lua's built-in patterns don't support lookahead.
158 |     -- We capture both the keyword and the TokenType, then filter out 'deprecated'.
159 |     -- Pattern breakdown:
160 |     --   %.%{           => matches ".{"
161 |     --   %s*"([~"]+)" => a quoted string -> capture 1
162 |     --   %s*,%s*TokenType%.([%w_]+) => TokenType.<Name> -> capture 2
163 |     local pat = "%.%{[%s*\"([~\"]+)\">%s*,%s*TokenType%.([%w_]+)"
164 |
165 |     local keywords = {}
166 |     for name, tok in content:gmatch(pat) do
167 |         if tok ~= "deprecated" then
168 |             keywords[#!keywords + 1] = name
169 |         end
170 |     end
171 |
172 |     table.sort(keywords)
173 |
174 |     vesti.print([[ \begin{tabular}{cccc} ]])
175 |
176 |     for i, kw in ipairs(keywords) do

```

```

177     local cell = string.format("\\keyword{%s}", kw)
178     if (i % 4) == 0 then
179         vesti.print(cell .. [[\\]])
180     else
181         vesti.print(cell .. "&")
182     end
183 end
184
185 vesti.print([[\\end{tabular}]]))
186 ::#[readAll]
187 \\caption{Keywords in Vesti}
188 }
189
190 \\subsection{\\keyword{docclass} keyword}
191 Keyword \\keyword{docclass} is an analogous of \\verb|\\documentclass| in \\LaTeX.
192 If \\keyword{docclass} is in the main paragraph, it acts just a normal word.
193 In other words, \\keyword{docclass} actives only in the preamble.
194 The syntax of \\keyword{docclass} is following:
195
196 useenv center {
197     \\keyword{docclass}\\ob<class name>\\ob{\\tt{<arguments>}{\\tt}}
198 }
199 Here, arguments are separated by commas and embraced by {\\tt ()}. Here are some
200 examples.
201
202 \\goodbreak
203 useenv itemize {
204     \\item \\keyword{docclass}\\ob{\\tt article}
205     \\item \\keyword{docclass}\\ob{\\tt article\\ob(10pt)}
206     \\item \\keyword{docclass}\\ob{\\tt article\\ob(10pt,\\ob twocols)}
207     \\item \\keyword{docclass}\\ob{\\tt article\\ob(10pt,twocols)}
208 }
209
210 \\subsection{\\keyword{importpkg} keywords}
211 Keyword \\keyword{importpkg} is an analogous of \\verb|\\usepackage| in \\LaTeX.
212 If \\keyword{importpkg} is in the main paragraph, it acts just a normal word.
213 In other words, \\keyword{importpkg} actives only in the preamble.
214
215 importpkg has two different syntax. First one is same as docclass.
216 useenv center {
217     \\keyword{importpkg}\\ob<pkg-name>\\ob{\\tt{arguments}{\\tt}}
218 }
219 Here, arguments are separated by commas and embraced by {\\tt ()}.
220 In the practical case, one should include several packages with options.
221 importpkg also supports such case. We will look at an example instead of
222 giving rigorous grammar.
223 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
224 +keyword|importpkg@ {
225     amsmath, amssymb, amsthm,
226     geometry (a4paper, margin=2.2cm),
227 }
228 -%}
229
230 \\noindent As one can see, inside of \\verb|{}|, several packages can be used
231 together with thier options.
232
233 \\subsection{\\keyword{startdoc} keyword}
234 Keyword \\keyword{startdoc} tells to Vesti that the main document starts. In the
235 main document, you can also write \\keyword{startdoc} in the main document. In
236 that case, \\keyword{startdoc} does nothing.

```

```

237 \subsection{\useenv keyword}
238 As the name implies, keyword \useenv is an analogous of \verb|\begin{...}| and
239 \verb|\end{...}| pair in \LaTeX.
240 The simplest \useenv is like this.
241
242
243 useenv figure [ht] {
244     \centering
245     useenv tikzpicture {
246         useenv scope {
247             \path (0,0) node {\vbox{
248                 %#\hbox{\tt\useenv center \{ }
249                 %#\hbox{\tt\obeyspaces Hello, World!}
250                 %#\hbox{\tt\obeyspaces\}}
251             } };
252         }
253         \path (2.3,0) node {or};
254         useenv scope [shift={(6,0)}]{
255             \path (0,0) node {\tt\useenv center \{ Hello, World! \}};
256         }
257     }
258 }
259
260 As you can see, {\tt\useenv center} is the part of \verb|\begin{center}|, and
261 the single {\tt\}} is the part of \verb|\end{center}|. Since Vesti knows their
262 pair, one can write a code with several environment, and each pair is properly
263 matched. For instance, above example is written in Vesti like follows. Here,
264 \verb|\useenv| just prints \useenv in that style.
265
266 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
267 |+color|purple@useenv@ figure [ht] {
268     \centering
269     |+color|purple@useenv@ tikzpicture {
270         |+color|purple@useenv@ scope {
271             \path (0,0) node {\vbox{
272                 |+color|blue@%#\hbox{\tt\useenv center \{ }
273                 |+color|blue@%#\hbox{\tt\obeyspaces Hello, World!}
274                 |+color|blue@%#\hbox{\tt\obeyspaces\}}
275             } };
276         }
277         \path (2.3,0) node {or};
278         |+color|purple@useenv@ scope [shift={(6,0)}] {
279             \path (0,0) node {\tt\useenv center \{ Hello, World! \}};
280         }
281     }
282 }
283 -%}
284
285 Full syntax about \useenv is the following.
286 useenv center {
287     \useenv\ob<environment name>\ob<argument>*\ob{\tt\{ } <body> {\tt\}}
288 }
289 where `*' means that the number of <argument> is zero or at least one, and
290 $$$
291     "<argument>" = useenv cases {
292         "<argument>" & "mandatory arguments" \cr
293         "[<argument>]" & "optional arguments" \cr
294     }
295 $$$
296 For instance, below one is a valid Vesti code (environment {\tt foo} is

```

```

297 undefined in general). As one can see, spaces cannot exist in between <argument>s.
298 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
299 |+color|purple@useenv@ foo (asd)(fff)[\ames and \awdsa](askws)[\rrsaa] {
300     foobar
301 }
302 -%}
303
304 #def #begenv {#::begenv(false,true)::#*}
305 #def #begenv_arg {#::begenv(#1,#2)::#*}
306 #def #endenv {#::endenv(false,true)::#*}
307 #def #endenv_arg {#::endenv(#1,#2)::#*}
308
309 \subsection{#begenv and #endenv keywords}
310 As the name implies, both keywords #begenv and #endenv are
311 analogous of \verb|\begin{...}| and \verb|\end{...}| pair in \LaTeX,
312 respectively. Thus below code
313 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
314 |+color|purple@begenv@ center
315     asdsad
316 |+color|purple@endenv@
317 -%}
318 is exactly same as
319 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
320 +keyword|useenv@ center {
321     asdsad
322 }
323 -%}
324 Then why we need #begenv and #endenv if we already have \useenv*?
325 The only reason which both #begenv and #endenv exist is
326 defining new environment. For instance, one defines a new environment like
327 following.
328 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@] {
329 %#+keyword|defenv@ foo { |+color|purple@begenv@ minipage(\textwidth)}{ |+color|purple@endenv@}
330 }
331
332 \subsection{\defun keyword}
333 \defun is the keyword which makes a \LaTeX\ function. Internally, it uses {\tt
334 xparse} package which is automatically included from modern \LaTeX$2\epsilon$
335 kernel. The grammar of \defun is the following.
336
337 begenv center
338     \defun\ob[<attr>]?\ob<command-name>\ob(<param-spec>)?\ob\{<implementation>\}
339 endenv
340 Here, {\tt`?`} means that this is an optional.
341
342 First, let us see some examples for \defun*, and then explain the meaning of
343 each fields.
344
345 #::
346 local examples = {
347     "\defun foo {Hello, World!}"
348 }
349
350 for _, example in ipairs(examples) do
351     vesti.print(vesti.parse(
352         "begenv! Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]"
353     ))
354     vesti.print(example)
355     vesti.print(vesti.parse("endenv! Verbatim"))
356 end

```

```

357 ::#
358
359 \section{Builtins}
360 Vesti also has its own builtin functions, which are prefixed with \#.
361 One might wonder what distinguishes builtins from keywords. In fact, from the
362 compiler's internal perspective, there is no real difference. However, in actual
363 language usage, constantly typing the prefix can be somewhat tedious, especially
364 for functions that are commonly used.
365
366 From the perspective of language design --particularly in Vesti-- it is
367 sometimes desirable to use names that can not be reserved as keywords. For example,
368 Vesti provides a built-in function \builtin|label|, which will be explained
369 later. Since Vesti is a typewriting-oriented language, the word \lq\lq
370 label\rq\rq\ is often used in its ordinary sense rather than in its special
371 semantic meaning within the language.
372
373 Followings are reserved as builtin functions.
374
375 useenv table [ht] {
376     \centering
377     #::
378     local content = read_all("../src/lexer/Token.zig")
379
380     -- match .{ "here" }
381     local pat = "%.{%s*\"([^\"]+)\">%s*%}"
382
383     local builtins = {}
384     for name, tok in content:gmatch(pat) do
385         builtins[#!builtins + 1] = name
386     end
387     table.sort(builtins)
388
389     vesti.print([[ \begin{tabular}{ccccc} ]])
390
391     for i, kw in ipairs(builtins) do
392         local cell = string.format("\\builtin@%s@", kw)
393         if (i % 5) == 0 then
394             vesti.print(cell .. [[ \\ ]])
395         else
396             vesti.print(cell .. "&")
397         end
398     end
399
400     vesti.print([[ \\end{tabular} ]])
401     ::#[readAll]
402     \caption{Builtins in Vesti}
403 }
404 Since {\tt fancyvrb} does not allow verbatim-like commands inside of
405 {\tt Verbatim} environment, builtins are not colored in example codes.
406
407 \subsection{\builtin|label| builtin}
408 For the compiling issue, if one label some environment, one should write like
409 follows using \LaTeX\ function \verb|\label|.
410 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=&|@]{%-
411 &keyword|useenv@ theorem { \label{eq:1}
412     1 + 1 = 2.
413 }
414 -%}
415
416 However, such code is not natural. Thus \builtin|label| comes into the place.

```

```

417 One can label environments using \builtin|label| builtin like this.
418 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=&|@]{%-
419 #label(eq:1) &keyword|useenv@ theorem {
420     1 + 1 = 2.
421 }
422 -%}
423 One can also write like the following, which I personally prefer.
424 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=&|@]{%-
425 #label(eq:1)
426 &keyword|useenv@ theorem {
427     1 + 1 = 2.
428 }
429 -%}
430
431 \subsection{\builtin|eq| builtin}
432 \builtin|eq| is a abbreviation of \lq\keyword{%-useenv-%} equation\rq.
433 Actually, below codes are same.
434
435 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
436 #label(eq:1)
437 +keyword|useenv@ equation {
438     \sum_{n=1}^{\infty} {1//n^2} = {\pi^2//6}.
439 }
440 -%}
441 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
442 #eq(eq:1) {
443     \sum_{n=1}^{\infty} {1//n^2} = {\pi^2//6}.
444 }
445 -%}
446 Since Vesti is used in the mathematical documents mainly, small syntactic suger
447 is needed, so Vesti introduce \builtin|eq| builtin.
448
449 \subsection{\builtin|chardef| builtin}
450 \builtin|chardef| defines a text token using the unicode codepoint.
451 Here is the grammar of \builtin|chardef| builtin.
452 useenv center {
453     \builtin|chardef|\ob<unicode-codepoint>\ob<function>
454 }
455 Here, <unicode-codepoint> must be hexadecimal.
456
457 \subsection{\builtin|mathchardef| builtin}
458 \builtin|mathchardef| defines a math token using the unicode codepoint.
459 Here is the grammar of \builtin|mathchardef| builtin.
460 useenv center {
461     \builtin|mathchardef|\ob<kind>\ob<font-num>\ob<unicode-codepoint>\ob<function>
462 }
463 Here, <unicode-codepoint> must be hexadecimal.
464 Before explaining each parameter, introduce some examples.
465 useenv itemize {
466     \item \builtin|mathchardef|\ob.opening\ob0\ob29d8\ob\verb|\lfooo|
467     \item \builtin|mathchardef|\ob.ordinary\ob0\ob2202\ob\verb|\diff|
468 }
469
470 \subsection{\builtin|enum| and \builtin|enum_counter| builtins}
471 \builtin|enum| builtin is just an enumerate environment with minimal support of
472 {\tt enumitem} feature. For example,
473 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
474 #enum {
475     \item aaa
476     \item bbb

```

```

477     \item ccc
478 }
479 -%}
480 prints
481 #enum {
482     \item aaa
483     \item bbb
484     \item ccc
485 }
486 \builtin|enum| also can be nested like follows.
487 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
488 #enum {
489     \item aaa
490     \item #enum {
491         \item bbb
492         \item ccc
493     }
494     \item ddd
495 }
496 -%}
497 It prints
498 #enum {
499     \item aaa
500     \item #enum {
501         \item bbb
502         \item ccc
503     }
504     \item ddd
505 }
506 To customize a format of \verb|\item| like {\tt enumitem}, one can
507 write like the following:
508 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
509 #enum (\Roman*. ) {
510     \item aaa
511     \item #enum ({\roman*}**) {
512         \item bbb
513         \item ccc
514     }
515     \item ddd
516 }
517 -%}
518 #enum (\Roman*. ) {
519     \item aaa
520     \item #enum ({\roman*}**) {
521         \item bbb
522         \item ccc
523     }
524     \item ddd
525 }
526 As one can see, \builtin|enum| changes the format with inside of {\tt()}.
527 In Vesti, single {\tt *} changes into {\tt \{<enum count>\}} (including braces),
528 and consecutive {\tt **} changes into single {\tt *} character.
529 Thus, \verb|\Roman*.| changes into \verb|\Roman{enumi}.| and \verb|{\roman*}**|
530 changes into \verb|{\roman{enumii}}*| (because this part is inside of another
531 \builtin|enum|).
532
533 Beware that by the parting mechanism, \verb|\roman***| changes into
534 \verb|\roman*<enum count>|, and \verb|\roman{*}**| into \verb|\roman{\{<enum count>\}}*|,
535 which are invalid arguments for \verb|\roman| \LaTeX\ internal command.
536

```

```

537 \builtin|enum_counter| builtin is a helper to get the name of the current enum
538 counter. For example, if one wants to start a second enum from 3, one can write
539 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
540 #enum {
541     \item aaa
542     \item #enum {
543         \setcounter{#enum_counter}{2}
544         \item bbb
545         \item ccc
546     }
547     \item ddd
548 }
549 -%}
550 and it prints
551 #enum {
552     \item aaa
553     \item #enum {
554         \setcounter{#enum_counter}{2}
555         \item bbb
556         \item ccc
557     }
558     \item ddd
559 }
560
561 \subsection{\builtin|get_filepath| builtin}
562 Because of the Vesti internal, if one includes picture by using
563 \verb|\includegraphics| for instance, providing the raw path cause an error from
564 \LaTeX\ with \lq\lq file not found\rq\rq. \builtin|get_filepath| builtin adjust
565 raw file path into the new relative one which \LaTeX\ knows. Here is the example
566 code.
567 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@] {
568 %#\includegraphics{#get_filepath(./foo.png)}
569 }
570 Internally, \builtin|get_filepath| changes file path into the relative one with
571 respect to {\tt.vesti-dummy}. For example, if {\tt foo.png}, {\tt foo.ves}
572 and {\tt.vesti-dummy} are located in the root directory, then inside of {\tt
573 foo.ves}, \builtin|get_filepath|{\tt(./foo.png)} will changed into
574 {\tt../foo.png} because for {\tt.vesti-dummy}, {\tt foo.png} is located outside
575 of it.
576
577 \subsection{\builtin|at_on| and \builtin|at_off| builtins}
578 Both builtins are same as \LaTeX\ functions \verb|\makeatletter| and
579 \verb|\makeatother| plus changes Vesti lexer such that \verb|@| character is
580 also can be a \LaTeX\ function name. For instance, without using \builtin|at_on|
581 command, below code does not compiles.
582 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|&] {
583 %#+keyword|defun& @foo (#1) {Hello, #1!}
584 }
585 This is because \keyword{% -defun-%} expects a valid \LaTeX\ function name but
586 the character \verb|@| is not a valid one except after the \verb|\makeatletter|
587 function. Since \keyword{% -defun-%} is a Vesti grammar, there is no way to tell
588 Vesti that \verb|@| is a right one although one uses \verb|\makeatletter|.
589 By using \builtin|at_on|, below code then compiles.
590 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|&]{%-
591 #at_on
592 +keyword|defun& @foo (#1) {Hello, #1!}
593 #at_off
594 -%}
595 One can not using \builtin|at_off|, but I strongly recommend to match
596 pairs.

```

```

597 \subsection{\buitin|ltx3_on| and \buitin|ltx3_off| builtins}
598 Those builtins are similar with \buitin|at_on| and \buitin|at_off|
599 pairs but for \LaTeX3.
600
601 \subsection{\buitin|textmode| and \buitin|mathmode| builtins}
602 Internally, vesti tracks so called {\tt text mode} and {\tt math mode}.
603 In the math mode, for instance, the token \verb|>| is changed into $->$ inside
604 of the math mode. The idea is that Vesti changes \verb|>| into \verb|\to|, and
605 so on. But when defining some function using \defun*, for example, one might
606 want to define
607 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
608 +keyword|useenv@ foo {-><-}
609 -%}
610 so that \verb|$ \foo$| makes $-><-$, one should change the mode. Here,
611 \buitin|mathmode| comes into the place. One should write instead that
612 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
613 +keyword|useenv@ foo {#mathmode{-><-}}
614 -%}
615
616 \section{Lua API}
617 Vesti uses Lua in both building script and inline inside of Vesti code.
618 Below are functions which are baked in Vesti compiler.
619 useenv table [ht] {
620     \centering
621     #::
622     local content = read_all("../src/Lua.zig")
623
624     local pat = '%.name%s*=%s*"([~"]+)"'
625
626     local lua = {}
627     for name, tok in content:gmatch(pat) do
628         lua[#lua + 1] = name
629     end
630     table.sort(lua)
631
632     vesti.print([[ \begin{tabular}{ccccc} ]])
633
634     for i, kw in ipairs(lua) do
635         local cell = string.format("\ttt{%s}", kw)
636         if (i % 4) == 0 then
637             vesti.print(cell .. [[\]])
638         else
639             vesti.print(cell .. "&")
640         end
641     end
642     end
643
644     vesti.print([[ \end{tabular} ]])
645     ::#[readAll]
646     \caption{Lua builtin functions in Vesti}
647 }
648
649 \newpage
650 \section{Source Code of This Document}
651 Below code was generated by inline lua.
652 useenv Verbatim [numbers=left, numbersep=5pt, frame=single] {
653 #::
654     local content = read_all("vesti_man.ves")
655     for line in content:gmatch("(^[~\r\n]*)\r?\n?") do

```

```
657         vesti.print(line)
658     end
659     ::#[readAll]
660 }
661
```